PROGRAMMENTWURF

AUFGABE

Entwickeln Sie ein Echtzeit-Spiel, welches auf der Konsole oder einer von Ihnen geschriebenen GUI läuft (Einbindung von externen Libraries ist erlaubt, darf aber nicht der Kern Ihrer Arbeit sein). Als Programmiersprache dürfen Sie zwischen Java und C++ wählen. Zentrale Eigenschaft Ihres Spiels muss sein, dass es sich um eine Multithreading-Anwendung handelt. Dies bedeutet nicht nur, dass die Anwendung mehrere Threads besitzt, sondern auch, dass die Nutzung von Nebenläufigkeit essenziell ist, das Spiel also ohne Multi-Threading nicht programmierbar wäre. Darüber hinaus sind Sie thematisch frei.

RAHMENBEDINGUNGEN

- Der Programmentwurf wird in Teams aus zwei oder drei Personen bearbeitet, die gemeinsam abgeben.
- Die Nutzung eines unterstützenden LLMs wie GitHub Copilot ist erlaubt.
- Sie erstellen den Programmentwurf teils in der Vorlesungszeit, teils in den 102 Stunden Selbststudium. Der Umfang Ihres Projekts sollte so gewählt sein, dass die Aufgabe in dieser Zeit lösbar ist.
- Sie dürfen Sich von der Aufgabenstellung des Programmentwurfs Software Engineering II des Jahrgangs TES21 inspirieren lassen. Der Umfang des damaligen Programmentwurfs wäre aber zu klein für Ihre Abgabe!
  Zum Vergleich: Die Studierenden aus TES21 hatten für Ihre damalige Aufgabe mit vorgegebenen Spielregeln und Requirements zwei Wochen Zeit. Sie müssen zwar Spielregeln und Requirements selbst erarbeiten, haben aber durch die semesterbegleitende Bearbeitung deutlich mehr Zeit.
- In der letzten Vorlesungseinheit stellen Sie als Team in einer 10-minütigen Präsentation Ihr Projekt vor, inkl. einer Live-Demo.
- Abgabe ist ~~eine Woche nach der Projektpräsentation~~ **am xx.4.2026**.

BESTANDTEILE IHRER ABGABE

Ihre Abgabe muss die folgenden Bestandteile enthalten:

1. Ein PDF-Dokument (oder mehrere), welches enthält
   - Eine kurze Beschreibung der Projektidee;
   - Eine Erläuterung der Regeln Ihres implementierten Spiels;
   - Eine Erläuterung des Multithreading-Charakters Ihrer Software, d.h. eine Auflistung der Threads im Programm mit kurzer Beschreibung der Funktionalität sowie eine Beschreibung der in mehreren Threads genutzten Ressourcen und wie der Zugriff kontrolliert wird, um inkonsistentes Verhalten auszuschließen;
   - Die von Ihnen erarbeiteten Requirements, die Ihre Software erfüllt;
   - Falls zutreffend: eine Erläuterung der verwendeten Designpatterns und Architekturmuster.
   - Antworten auf die unten genannten „Fragen an Ihr entwickeltes Spiel zum Thema Verlässlichkeit“
2. Den Quellcode Ihrer Software, welcher in Java oder C++ geschrieben ist und fehlerfrei kompiliert.

FRAGEN AN IHR ENTWICKELTES SPIEL ZUM THEMA VERLÄSSLICHKEIT

Beantworten Sie in Ihrer Abgabe die folgenden Fragen zum Thema Verlässlichkeit. Die Seitenangaben beziehen sich auf den Foliensatz zum Thema Verlässliche Systeme:

1. Welche Folgen (vgl. Foliensatz Seite 7) könnte ein Systemausfall Ihres Spiels haben? Wie kritisch bewerten Sie dies?
2. Welche Mechanismen haben Sie verwendet, um die Verlässlichkeit Ihres Spiels zu erhöhen?
3. Welche Metrik(en) für Zuverlässigkeit und Verfügbarkeit (vgl. Foliensatz Seite 44 ff) ist/sind Ihrer Meinung nach für Ihr Spiel am besten geeignet? Welche Wahrscheinlichkeits-Werte würden Sie verlangen?
4. Beschreiben Sie qualitativ, wie Sie einen Testdatensatz (vgl. Foliensatz Seite 65 ff) erstellen würden um nachzuweisen, dass Ihre Metrik den gewählten Wahrscheinlichkeits-Wert einhält.

Für Frage 3 und 4 sollten Sie Vergleichswerte recherchieren. Nennen Sie Ihre Quellen und begründen Sie, weshalb die Vergleichswerte auf Ihr Spiel übertragbar sind oder weshalb Sie die eigenen Werte anders gewählt haben!

BEWERTUNGSKRITERIEN:

Punkte	Kriterium
5	Im PDF: Originalität der Projektidee, passender Umfang des Projekts
10	Im PDF: Erklärung der Spielregeln, Erläuterung von Architekturmustern oder Designpatterns, Beschreibung wichtiger Aspekte Ihrer Software (außer Multithreading-Eigenschaften)
10	Im PDF: Die Multithreading-Eigenschaften des Spiels werden beschrieben, d.h. welche Threads es gibt, welche Ressourcen gemeinsam genutzt werden und wie inkonsistentes Verhalten durch nebenläufigen Zugriff verhindert wird.
10	Im PDF: Die Requirements sind konsistent, vollständig, realisierbar und verifizierbar und bilden geeignet die Spielregeln sowie weitere funktionale und nicht-funktionalen Anforderungen an das Spiel ab.
10	Im PDF: Die Fragen zum Thema Verlässlichkeit werden beantwortet.
10	Im Quellcode: Das Programm ist in Java oder C++ geschrieben und kompiliert fehlerfrei. Das Spiel ist über die Konsole oder eine geeignete GUI spielbar.
15	Im Quellcode: Das Programm enthält mindestens drei Threads. Gemeinsam genutzte Ressourcen sind geeignet geschützt vor Inkonsistenzen durch asynchronen Zugriff.
10	Im Quellcode: Das Projekt ist gut strukturiert, leicht zu lesen, verwendet verständnisfördernde Benennungen und ist ausreichend kommentiert. Das Projekt folgt den Richtlinien von Clean Code.
Bonus: 10	Für eine sehr gute Präsentation oder überdurchschnittlichen Spielspaß oder einen sehr großen Projektumfang oder eine besonders gute PDF-Dokumentation sind je Kategorie 2-3 Bonuspunkte möglich, in Summe maximal 10 Bonuspunkte.

Viel Erfolg!